

Table of Contents

Deep Open Information Extraction from Unstructured Data (DOIEUD).....	2
1 Abstract.....	2
2 Introduction.....	2
3 Background.....	3
3.1 Baselines	4
4 Methodology	5
4.1 Proposed Method.....	5
4.1.1 Data Preprocessing.....	5
4.1.2 Data Preparation	6
4.1.3 Model Architecture	12
4.2 Experiments	16
4.2.1 Experimental Setup.....	16
4.2.2 Experimental Results.....	18
5 Conclusion.....	26
6 References	27
Figure 1 Overview of system architecture (DOIEUD)	5
Figure 2 SubjectObjectrelation Flow Diagram	9
Figure 3 functionTriples Flow Diagram	10
Figure 4 Overview of Data Preparation. Given the sentence: “Joseph Robinette Biden Jr.	11
Figure 5 Overview of details within rows	12
Figure 6 Load Data.....	12
Figure 7 Initialize Model.....	13
Figure 8 tokenizer.....	14
Figure 9 Train Model.....	15
Figure 10 Overview of system architecture (DOIEUD) Given the sentence: 'Philip plays the..... '	16
Figure 11 Similarity with sentences	18
Figure 12 wire75 DOIEUD (1) Avg:4.23612557 Min:3.2511247.....	25
Figure 13 wire75 DOIEUD (2) Avg:3.55247976 Min:1.5615752	25
Figure 14 CARB DOIEUD (1) Avg:2.366318955 Min:1.701424.....	25
Figure 15 CARB DOIEUD (2) Avg:1.684737347 Min:0.7057095.....	25
Table 1: The statistical datasets.	16
Table 2 Number of Sentences extracted relation	18
Table 3 similarity with sentences	18
Table 4 Illustration of annotation principles. The facts in red (preceded by a cross mark) should not.	19
Table 5 Performances of OpenIE systems on the BenchIE and BenchIE ^{FL} dataset.....	22
Table 6 Performance of OpenIE systems on Wire57, OIE (benchie), Re-OIE2016, CDR and CaRB datasets.	24

Deep Open Information Extraction from Unstructured Data (DOIEUD)

1 Abstract

The paper explores the integration of cutting-edge natural language processing techniques, specifically BERT, Named Entity Recognition (NER), and Dependency Parsing with spaCy, to perform deep open information extraction from unstructured data. The study showcases the enhanced capabilities of these models in extracting structured information from raw text, demonstrating their effectiveness in handling complex information extraction tasks. The study explores using advanced natural language processing techniques like BERT, Named Entity Recognition, and Dependency Parsing with spaCy to extract structured information from unstructured data. Results show high accuracy in extracting relationships from text compared to other models. The DOIEUD (Deep Open Information Extraction from Unstructured Data) model performs OpenIE systems significantly, especially when using the AdamW optimizer. Lower loss values indicate better performance of the model on training data.

2 Introduction

Information Extraction (IE) is a crucial task in Natural Language Processing (NLP) that involves automatically extracting structured information from unstructured text data. Unstructured text refers to data that does not have a predefined data model or is not organized in a pre-defined manner, such as emails, social media posts, news articles, and research papers. IE aims to transform this raw textual data into a structured format that can be easily analyzed and interpreted by machines. In this context, recent advancements in NLP, particularly the advent of pre-trained language models like BERT, have revolutionized the field by enabling deep contextual understanding of text.

Significance of Deep Learning Models like BERT and NER for Advanced NLP Tasks:

Deep Learning models have significantly transformed the landscape of NLP tasks, including Information Extraction. Two key models that have played a pivotal role in advancing NLP capabilities are BERT (Bidirectional Encoder Representations from Transformers) and Named Entity Recognition (NER).

BERT is a pre-trained transformer-based model that has revolutionized NLP tasks by capturing bidirectional context in text. It excels in understanding the nuances and context of language, making it highly effective for tasks such as text classification, question answering, and information extraction. BERT's ability to contextualize words based on their surrounding context has significantly enhanced the accuracy and performance of IE systems.

Named Entity Recognition (NER) is a fundamental task in NLP that involves identifying and categorizing named entities within text, such as names of people, organizations, locations, dates, and more. NER plays a crucial role in information extraction by enabling systems to recognize and extract specific entities of interest from unstructured text. Deep Learning models, including BERT, have greatly improved the accuracy of NER systems, leading to more precise entity extraction and better information retrieval.

Role of spaCy in NER Extraction and Dependency Parsing:

spaCy is a popular open-source NLP library in Python that provides efficient tools for NLP tasks such as Named Entity Recognition (NER), part-of-speech tagging, dependency parsing, and more.

In the context of NER extraction and dependency parsing:

NER Extraction: spaCy offers pre-trained models for NER that can identify and classify named entities in text. By leveraging spaCy's NER capabilities, developers and researchers can easily extract entities

like persons, organizations, locations, and more from unstructured text data. The accuracy and performance of spaCy's NER models make it a valuable tool for information extraction tasks.

Dependency Parsing is the process of analyzing the grammatical structure of a sentence to determine the relationships between words. spaCy provides efficient algorithms for dependency parsing that help in understanding the syntactic structure of text. By parsing dependencies, researchers can extract valuable insights about how different elements in a sentence are related, aiding in tasks like information extraction and text analysis.

Contributions:

- This research contributes to the existing body of knowledge by showcasing a comprehensive approach that integrates cutting-edge NLP techniques for deep open information extraction. The findings from this study aim to provide insights into the enhanced capabilities of these models in handling complex information extraction tasks.
- We propose Open Information Extraction (OIE) for extracting useful relation representations directly from unstructured text based on the combination of deep learning models like BERT, NER techniques, and tools like spaCy for NER extraction and dependency parsing has significantly advanced the field of Information Extraction from unstructured text, enabling more accurate and efficient processing of textual data for a wide range of applications. Our method can handle more complex document-level data without sacrificing model accuracy compared to the currently most used joint modeling methods.

3 Background

Information Extraction involves the task of identifying and extracting structured information from unstructured text sources. Challenges such as ambiguity, context-dependency, and variability in language usage make this process intricate. BERT (Bidirectional Encoder Representations from Transformers) has emerged as a powerful tool for capturing contextual information, enabling more accurate and context-aware information extraction.

Overview of BERT for Contextual Understanding:

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based deep learning model that has revolutionized Natural Language Processing (NLP) tasks by capturing bidirectional context in text. Unlike traditional language models that process words in a sequential manner, BERT is designed to understand the context of each word in relation to its surrounding words bidirectionally. This bidirectional approach allows BERT to capture complex linguistic patterns and dependencies, leading to better contextual understanding of text.

Key features of BERT include:

Pre-training on large corpora: BERT is pre-trained on vast amounts of text data, enabling it to learn general language representations that can be fine-tuned for specific tasks.

Bidirectional context: BERT considers both left and right context when processing a word, allowing it to capture deeper semantic relationships.

Transformer architecture: BERT utilizes a transformer architecture that enables parallel processing of words, making it efficient for capturing long-range dependencies in text.

BERT has been widely adopted in various NLP tasks, including:

Named Entity Recognition: BERT enhances Named Entity Recognition (NER) tasks by providing contextual information that aids in accurately identifying and categorizing entities in text.

Importance and Applications of Named Entity Recognition (NER) in Extracting Entities:

Named Entity Recognition (NER) is a fundamental NLP task that involves identifying and classifying named entities within text into predefined categories such as names of people, organizations, locations, dates, and more. The importance of NER lies in its ability to extract specific entities of interest from unstructured text, enabling machines to understand and process textual information more effectively.

Applications of NER include:

Information Extraction: NER plays a crucial role in extracting structured information from unstructured text data. By identifying named entities, NER systems facilitate the extraction of key information such as names of people, organizations, and locations.

Relation Extraction: NER is often used as a precursor to relation extraction tasks where the relationships between entities in text are identified and analyzed.

Search and Recommendation Systems: NER is utilized in search engines and recommendation systems to improve the relevance of search results and recommendations by understanding and categorizing entities mentioned in queries or documents.

Role of Dependency Parsing in Understanding Text Structure:

Dependency parsing is a syntactic analysis technique used in NLP to analyze the grammatical structure of sentences by determining the relationships between words. By representing the dependencies between words in a sentence as a tree structure, dependency parsing provides valuable insights into the syntactic relationships and hierarchical structure of text.

The role of dependency parsing includes:

Text Structure Analysis: Dependency parsing helps in understanding the syntactic structure of text by identifying the relationships between words, such as subject-verb relationships, object dependencies, and modifiers.

Information Extraction: Dependency parsing aids in extracting structured information from text by revealing the syntactic dependencies between words. This information is valuable for tasks like entity extraction, relation extraction, and event extraction.

Machine Translation: Dependency parsing is used in machine translation systems to align words and phrases in the source and target languages based on their syntactic dependencies, improving translation accuracy.

In summary, BERT's contextual understanding, NER's entity extraction capabilities, and dependency parsing's role in text structure analysis are essential components in advancing NLP tasks, enabling more accurate and nuanced processing of unstructured text data for a wide range of applications.

3.1 Baselines

As baseline models, Open information extraction systems have evolved from rule-based systems like ReVerb [1], ClausIE [2] or MinIE [3], which use parts-of-speech tags and syntactical analysis to derive extraction rules. Newer systems, like ImojIE and M2OIE, use neural approaches and sequence-to-sequence or sequence tagging techniques to output the original sentence and delimit arguments and relations. Sequence-to-sequence models like ImojIE [6] and M2OIE [7] output the original sentence word at a time, while neural tagging models like OpenIE6 [5] and CompactIE [8] focus on identifying relations and tagging related words. Both models are trained on bootstrapped data from previously released extractors.

This paper [4] explored the creation of relation representations from text using extensions of Harris' distributional hypothesis and BERT. The authors demonstrated that these representations outperform previous models for relation extraction tasks. They introduced a new training method called "matching the blanks" which, combined with a new architecture for fine-tuning relation representations in BERT, achieves top results in relation extraction tasks. The models even surpass human accuracy in few-shot relation matching. The study highlighted the effectiveness of the new model in low-resource scenarios and suggests it can reduce the human effort required for building relation extractors.

Graph4NLP¹ is an open-source library that facilitates research in graph neural networks for Natural Language Processing (NLP). It offers full implementations of state-of-the-art models for various NLP

¹ The codes and details of Graph4NLP library are provided at <https://github.com/graph4ai/graph4nlp>.

applications, including text classification, semantic parsing, machine translation, KG completion, and natural language generation. Graph4NLP also provides flexible interfaces for building customized models and is built on optimized runtime libraries like DGL and Pytorch [9].

BenchIE^{FL} [11] is the newest version of the OIE benchmark that implements the principles of BenchIE [10] more effectively, with fewer errors and shortcomings when comparing candidate facts to reference ones. It enables more accurate conclusions about the performance of OIE extractors by improving annotations and introducing a new matching function for more valid extractions.

4 Methodology

Neural Open Information Extraction (Neural OpenIE) refers to the use of neural network models to perform the task of extracting structured information (such as subject-relation-object triples) from unstructured text. Unlike traditional rule-based or pattern-based approaches, neural models can learn representations directly from data, which can lead to improved performance in capturing complex relationships and handling diverse linguistic patterns.

OpenIE task categorize into three groups: Open Relation Triplet Extraction (ORTE), Open Relation Span Extraction (ORSE) and Open relation clustering (ORC).

- ORTE Task: Text $\rightarrow$ Relational Triplet
- ORSE Task: Entities + Text $\rightarrow$ Relation Span
- ORC Task: Entities + Text $\rightarrow$ Clustering without Explicit Relation Span or Label Open relation clustering (ORC)

4.1 Proposed Method

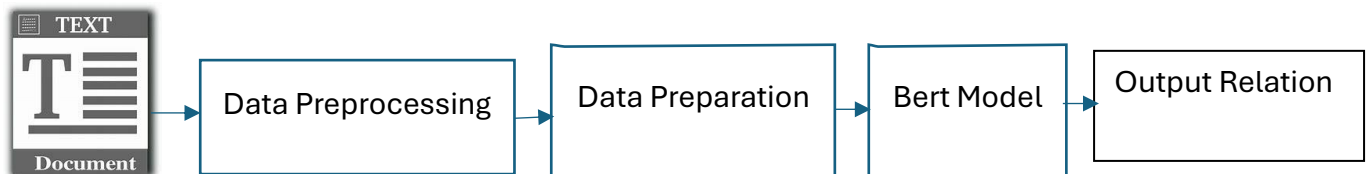


Figure 1 Overview of system architecture (DOIEUD)

In this section we describe our system, Deep Open Information Extraction from Unstructured Data (DOIEUD) see Figure 1 and Figure 10 Overview of system architecture (DOIEUD) Given the sentence: 'Philip plays the bass, because he loves it'.. based on the group of **ORTE** and **ORSE**. We first detail the data preprocessing, data Preparation and Model Architecture.

4.1.1 Data Preprocessing

1. Pre-process special characters in sentences
2. Contractions are shortened forms of words or combinations of words, where letters are dropped and replaced by an apostrophe. For example, "I'll" expands to "I will," and "shouldn't" expands to "should not." By using contractions.fix, you can convert contractions back to their full forms, which can be helpful for text pre-processing in natural language processing (NLP) tasks.
3. Splits concatenated words using natural language processing (NLP) based on English Wikipedia using wordninja
4. Removes spaces between numbers
5. Removes spaces between word inside ()[]
6. For example: In brain membranes from spontaneously hypertensive rats clonidine, 10 (- 8) to 10 (- 5) M , did not influence stereoselective binding of [3H] - naloxone (8 nM) , and naloxone

, 10 (- 8) to 10 (- 4) M , did not influence clonidine - suppressible binding of [3H] - dihydroergocryptine (1 nM).→In brain membranes from spontaneously hypertensive rats clonidine,10(-8)to10(-5)M, did not influence stereoselective binding of [3H]-naloxone (8NM), and naloxone,10(-8)to10(-4)M, did not influence clonidine-suppressible binding of [3H]-dihydroergocryptine (1NM).

7. The document is split **into sentences boundary detection** using **chunkipy -TextChunker**.
 - a. Chunkipy is an extremely useful Python library for segmenting long texts into smaller chunks based on either a character or token count. It's particularly handy for tasks like natural language processing (NLP) and information extraction. Here are some key features:
 - b. Customizable Chunk Sizes: You can specify the desired chunk size=500.
 - c. Splitting Strategies: chunkipy provides flexibility by allowing you to choose from predefined splitting strategies or define your own.
 - d. Token Estimation: Unlike other text chunking libraries, chunkipy considers tokenization, making it compatible with token-based models like BERT.
 - e. Smart Overlapping: You can create overlapping chunks to preserve context.
 - f. Sentence Preservation: By default, chunkipy avoids cutting sentences, ensuring syntactically correct chunks. For example, if you have a long text with more tokens than BERT's input size, chunkipy can help you split it effectively.

4.1.2 Data Preparation

Neural models require a large amount of annotated data for training. You'll need a dataset where each sentence is annotated with the relevant triples (e.g., subject, relation, object).

1. Given a sequence of tokens $X = (x_1, x_2, \dots, x_n)$,
2. Coreference resolution is used to resolve references to previous entities the library `allennlp.predictors.predictor`. For example: Cluster '[Philip; he]' [the bass; it]



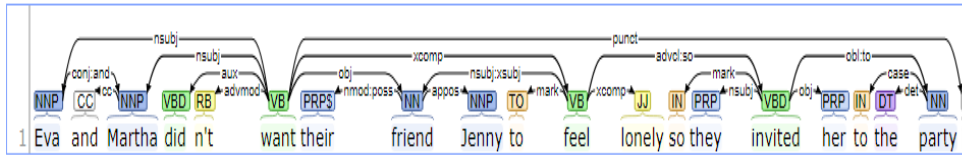
3. NER Extraction: using spacy
 - a. Using `nlp=spacy.load("en_core_web_lg")`
 - b. `nlp.add_pipe("merge_entities")`
 - c. `nlp.add_pipe("merge_noun_chunks")`
 - d. `entities_of_interest = ["PERSON", "NORP", "FAC", "ORG", "GPE", "LOC", "PRODUCT", "EVENT ", "WORK_OF_ART", "LAW", "LANGUAGE", "DATE"]`

Named Entity Recognition:

	PERSON	PERSON		PERSON
1	Eva and Martha did n't want their friend Jenny to feel lonely so they invited her to the party .			

4. Dependency Parser using spaCy

Enhanced++ Dependencies:



The provided code is a Python function called SubjectObjectrelation that takes a list of tokens (presumably from a natural language processing library like spaCy) and extracts subject-relation-object (SRO) triples from the input. Here's a detailed explanation of the code's functionality and flow (see Figure 2 SubjectObjectrelation Flow Diagram): The key components and functionality of the code are as follows:

- a. Initialize Variables: The function starts by initializing various lists and variables, including:
 - SUBJECTS = ["subj", "nsubj", "nsubjpass", "csubj", "csubjpass", "agent", "expl"]
 - OBJECTS = ["dobj", "dative", "attr", "oprd", "pobj", "appos", "nummod", "compound", "npadvmod", "advmod", "mod"]
 - ADJECTIVES = ["ROOT", "prep", "acomp", "advcl", "amod", "nn", "nmod", "ccomp", "complm", "adj", "agent", "ccomp", "advcl", "relcl", "hmod", "infmod", "xcomp", "rmod", "poss", "possessive", "aux", "neg"]
 - tagRelation=["VBD", "VB", "VBG", "VBN", "VBP", "VBZ", "MD", "RB", "JJ", "JJR", "JJS", "RBR", "RBS", "RP"]
 - tagnotsubject=["WP", "VBD", "IN", "WDT", "WP\$", "WRB", "VBD", "VB", "VBG", "VBN", "VBP", "VBZ", "DT", "RB", "JJ", "JJR", "JJS", "RBR", "RBS", "RP"] are used to identify the different components of the subject-relation-object triples.
 - b. Token Processing: The function then iterates through each token in the input. For each token, it performs the following steps:
 - Skip Punctuation: If the current token is a punctuation mark, it is skipped.
 - Check Subject: The function checks if the current token is a subject based on its dependency tag (SUBJECTS) and part-of-speech tag (tagsubject). If it is, the subject variable and subjectindex are updated accordingly, handling cases where the subject is a multi-word phrase or has conjunctions.
 - Check Object: The function checks if the current token is an object based on its dependency tag (OBJECTS) and part-of-speech tag (tagsubject). If it is, the object variable and objectindex are updated accordingly, handling cases where the object is a multi-word phrase or has conjunctions.
 - Check Relation: The function checks if the current token is a relation based on its dependency tag (ADJECTIVES) and part-of-speech tag (tagRelation). If it is, the relation variable and relIndex are updated accordingly, handling cases where the relation is a multi-word phrase.
 - Call function Triples: The function calls the functionTriples function (show in Figure 3 functionTriples Flow Diagram) with the current token's subtree and extracts subject-relation-object (SRO) triples from the input text.
- The key components of this code are:
- Identifying subjects, relations, and objects from the input text
 - Forming SRO triples and checking for uniqueness
 - Maintaining and restoring the main subject, relation, and their indices
 - Extracting the relevant tokens for each identified triple

- c. Missing Object Handling: The function checks for cases where the object is empty, the relation is not empty, and the token is not in the subject. In such cases, it removes the token from the relation and appends it to the object, updating the objectindex accordingly.
- d. Extract Triple: After processing all the tokens, the function checks if a valid subject-relation-object triple has been extracted.
 - Check Triple Validity: The function checks if the extracted triple is valid (i.e., the subject, relation, and object are not empty, and the subject is different from the object) and not a duplicate.
 - Add Triple: If the triple is valid and not a duplicate, it is added to the triple list.
 - Update Tokens: The function updates the rtoken1 list with the relevant tokens for the extracted triple.
 - Reset Variables: The function resets the object, relation, and subject variables to prepare for the next iteration.
- e. Output: The function returns the triple list (containing the extracted subject-relation-object triples) and the rtoken1 list (containing the corresponding token sequences).

The code uses various NLP techniques, such as dependency parsing and part-of-speech tagging, to extract the necessary information from the input text.

The overall purpose of this code is to extract subject-relation-object triples from the input tokens and maintain relevant information about the extracted triples, such as the token indices for the subject, relation, and object. The extracted triples and token sequences can be used for further analysis or processing in a natural language processing pipeline. The code handles various edge cases, such as multi-word phrases, conjunctions, and duplicate triples, to ensure the extracted triples are accurate and meaningful.

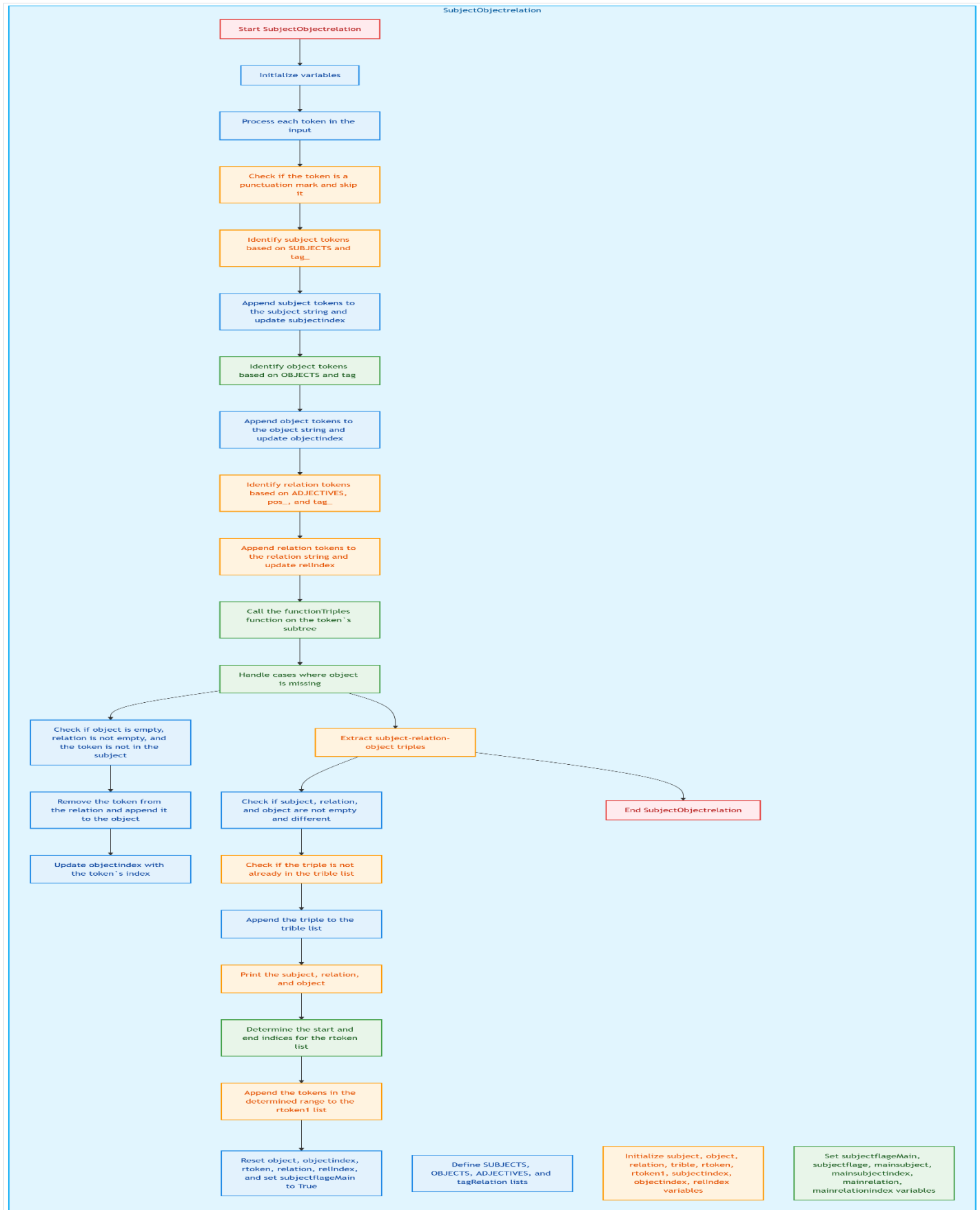


Figure 2 SubjectObjectrelation Flow Diagram

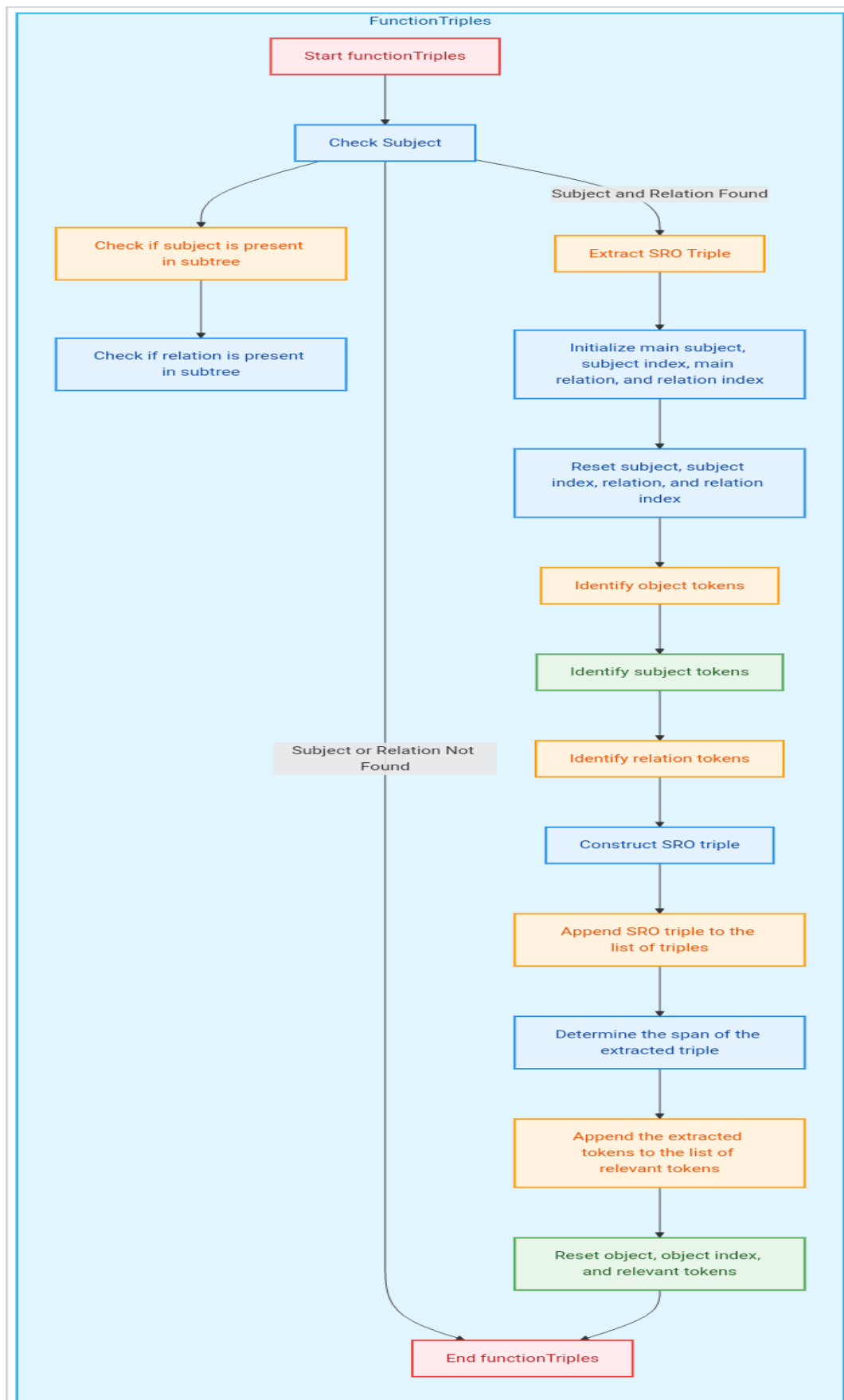


Figure 3 functionTriples Flow Diagram

We follow their 4-step to obtain Data Preparation (input token, subject, object, relation between them) the process shown in Figure 4 Overview of Data Preparation. Given the sentence: “Joseph Robinette Biden Jr. is an American politician who is the 46th and current president of the United States. A member of the Democratic Party, he served as the 47th vice president from 2009to2017 under Barack Obama and represented Delaware in the United States Senate from 1973to2009.” and Figure 5 Overview of details within rows.



Figure 4 Overview of Data Preparation. Given the sentence: “Joseph Robinette Biden Jr. is an American politician who is the 46th and current president of the United States. A member of the Democratic Party, he served as the 47th vice president from 2009to2017 under Barack Obama and represented Delaware in the United States Senate from 1973to2009.”

a sentence sequence	['Joseph Robinette Biden Jr.', 'is', 'an American politician', 'who', 'is', 'the 46th and current president', 'of', 'the United States', '.']	['Joseph Robinette Biden Jr.', 'is', 'an American politician', 'who', 'is', 'the 46th and current president', 'of', 'the United States', '.', 'A', 'member', 'of', 'the Democratic Party', '.']
Position of subject	(0, 1)	(0, 1)
Position of object	(7, 8)	(12, 13)
Relation extraction from function using Dependency Parser/NER extraction	'is an American politician is the 46th and current president, the 46th and current president of'	
Position of relation	[1, 2, 4, 5, 5, 6]	
Subject	'Joseph Robinette Biden Jr.'	'Joseph Robinette Biden Jr.'
object	'the United States'	'the Democratic Party'

Figure 5 Overview of details within rows

4.1.3 Model Architecture

Load Data

1. LoadDataLoaders: Loads the training data, preprocesses and tokenizes it, and creates PyTorch DataLoader objects.
2. LoadTokenizer: Loads the tokenizer from a pickle file and retrieves the special tokens (e1, e2, pad, mask) the process shown in Figure 6 Load Data.

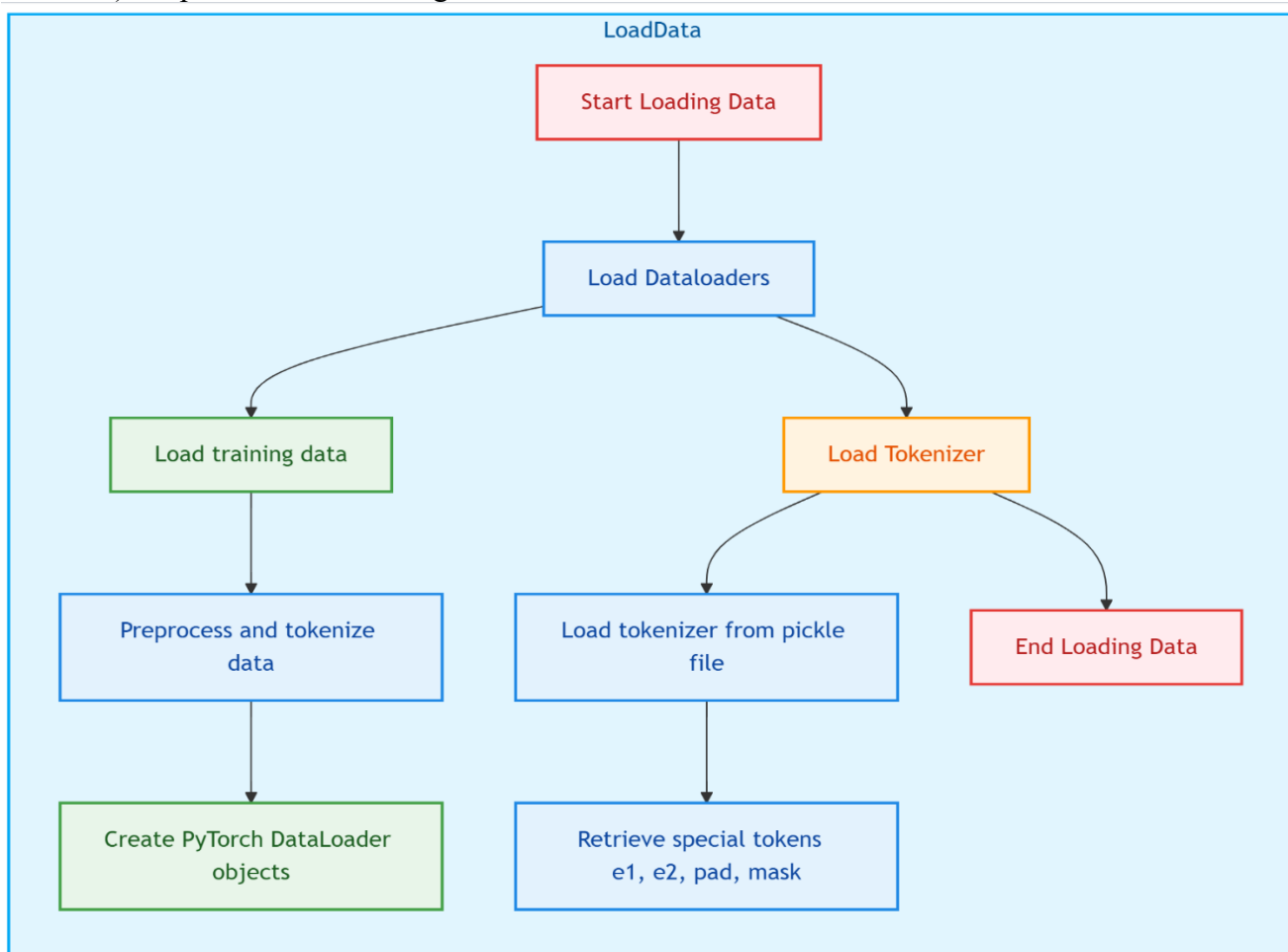


Figure 6 Load Data

Initialize Model

1. LoadModel: Loads the pre-trained BERT model and resizes the token embeddings to match the tokenizer.
 2. InitModel: Sets the model to training mode and moves it to the GPU if available.
 3. InitLoss: Creates the Two_Headed_Loss object for the training loss function.
 4. InitOptimizer: Creates the (AdamW or Adam) optimizer and the learning rate scheduler.
 5. LoadCheckpoint: Loads the previous training state, losses, and accuracy from a checkpoint.
- the process shown in Figure 7 Initialize Model.

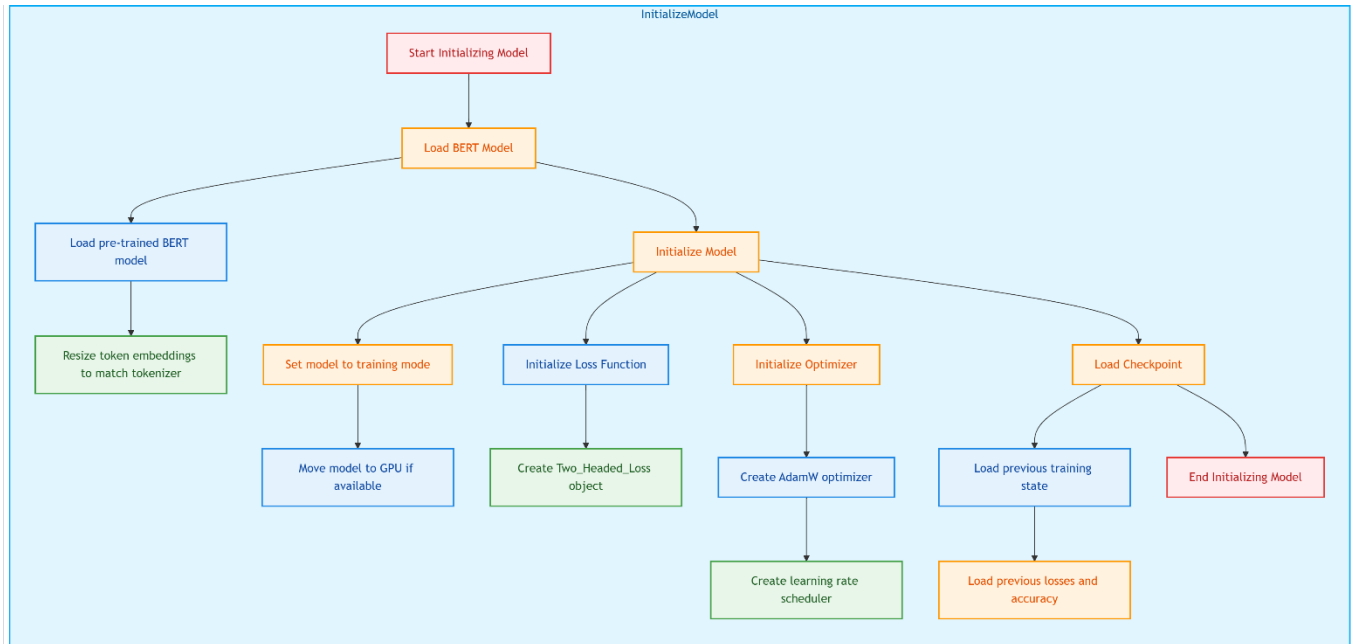


Figure 7 Initialize Model

Train Model

1. Train Epoch: Trains the model for one epoch, including the following steps:
 - a) Tokenize Data (the process shown in Figure 8 tokenizer):
 - i) from Section 2, that the triple statement $D = (x, s1, s2, r, relIndex)$, $e1, e2$ contains the sequence of tokens x and the entity span identifiers $s1$ and $s2$, the relation extracted r and the relation index $relIndex$
 - ii) Relation Masking
 - iii) Given a sequence of tokens $X = (x1, x2, \dots, xn)$, and modify x to $\tilde{x} = [x0, x1 \dots [mask] \dots xn]$ the $[mask]$ indices to the relation index $relIndex$ or $masked_idxs$ random if $relIndex$ contains no values
 - iv) Entity marker token, we augment x with four reserved word pieces to mark the begin and end of each entity mentioned in the relation statement. We introduce the $[E1]$, $[/E1]$, $[E2]$ and $[/E2]$ and modify $\tilde{x}$ to give $x1 \sim = [x0 \dots [E1] xi \dots xj-1 [/E1] \dots [E2] xk \dots xl-1 [/E2] \dots xn]$. and we feed this token sequence into BERT instead of x . We also update the entity indices $\tilde{s1} = (i + 1, j + 1)$ and $\tilde{s2} = (k + 3, l + 3)$ to account for the inserted tokens.
 - v) Input Representation: Convert input sentences into a format suitable for the neural network model. This often involves tokenization, encoding, and optionally, attention masks to handle variable-length sequences.

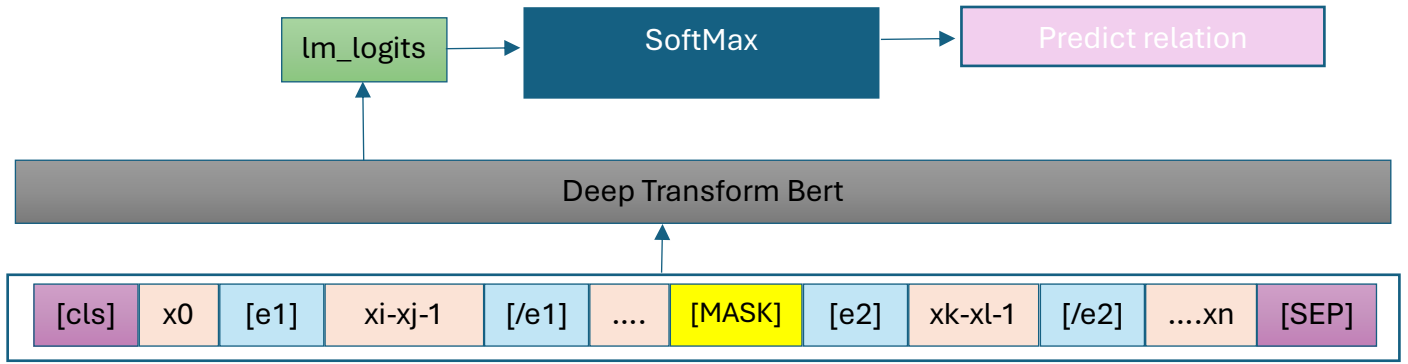


Figure 8 tokenizer

- b) Computes accuracy, the loss and backpropagates the gradients.
 - c) Clips the gradients to prevent exploding gradients.
 - d) Updates the model parameters.
 - e) Computes and logs the training metrics.
 - f) Updates the learning rate scheduler.
2. Train Epoch End: Performs the following actions at the end of each epoch:
 - a) Computes the average loss and accuracy for the epoch.
 - b) Saves the epoch-level metrics.
 - c) Saves a model checkpoint.
 - d) Updates the best model if the accuracy has improved.

the process shown in Figure 9 Train Model.

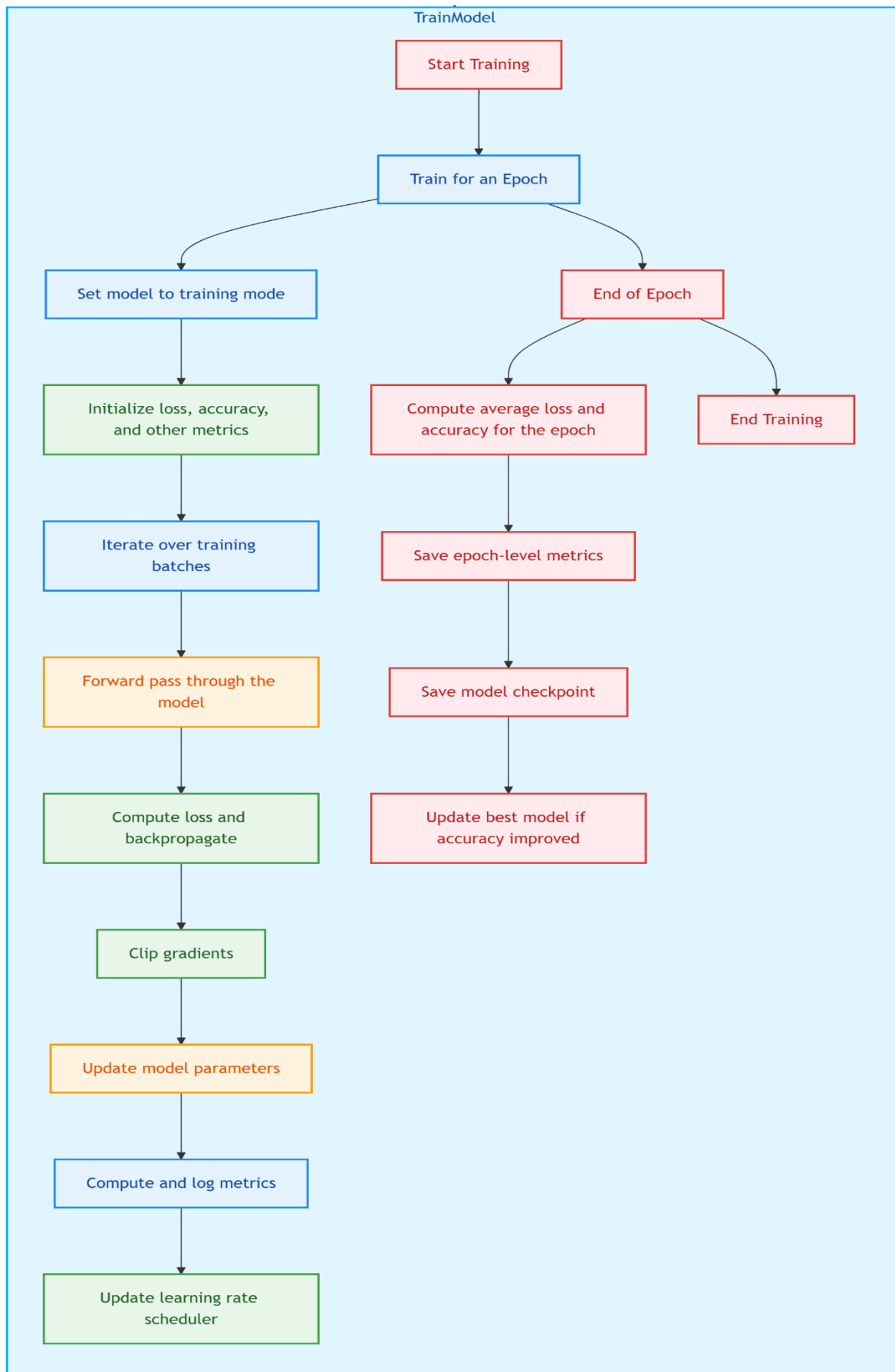


Figure 9 Train Model

Figure 10 shows an overview of DOIEUD architecture.

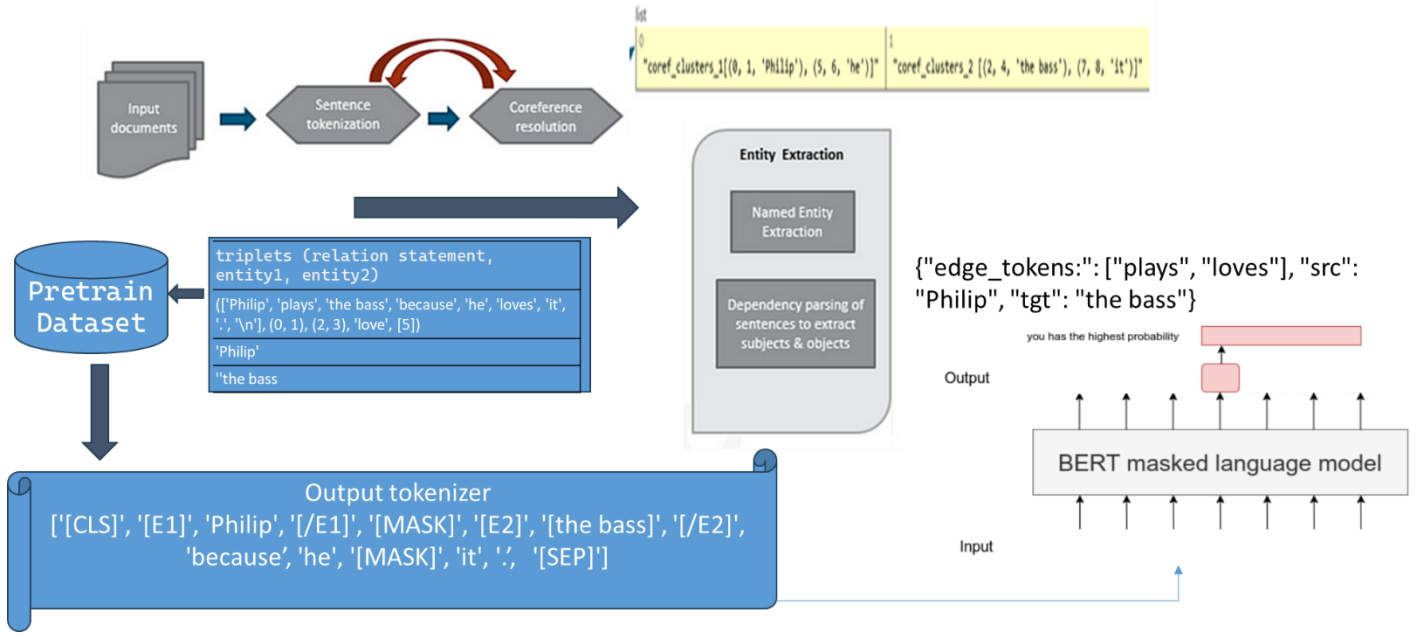


Figure 10 Overview of system architecture (DOIEUD) Given the sentence: 'Philip plays the bass, because he loves it'.

4.2 Experiments

4.2.1 Experimental Setup

Datasets

We evaluate our DOIEUD model on benchmarks widely used datasets shown in

Table 1: The statistical datasets.:

- WiRe57 ^[2]: This benchmark is comprised of a small, expertly annotated corpus of 57 sentences and uses a token-level matching function.
- CaRB ^[3]: This benchmark, which is a crowd-sourced annotation of 1200 sentences, also uses a token level scoring function. We used the test set which is made up of a total of 641 sentences.
- BenchIE ^[4]: This benchmark uses clusters or synsets to group all formulations of a single fact, allowing usage of an exact matching function. It is made up of 300 manually annotated sentences. We used the default facet.
- Re-OIE2016 ^[5]: Re-OIE2016.json is the relabeled dataset of OIE2016 in a Json format. Note that the sentence order in Re-OIE2016 is different in that of OIE2016 but the sentences are almost the same.
- CDR ^[6]: This benchmark uses Bio-DocuRE.

Table 1: The statistical datasets.

	wire 57	OIE	Re-OIE2016	carb	CDR
Original Dataset	57	300	591	641	500
Original Triples	325	1354	1724	2715	-

² <https://github.com/rali-udem/WiRe57>

⁵ https://github.com/zhanjunlang/Span_OIE/blob/master/evaluate/Re-OIE2016.json

³ <https://github.com/dair-iitd/CaRB>

⁶ <https://github.com/luguo-ry/filr>

⁴ <https://github.com/gkirl/benchie>

https://drive.google.com/drive/folders/13RgVm7IfEnm4_dV2UTQKIwJExlhyD4pA

Evaluation

In OpenIE, the input sentence is not restricted to any domain, and the extraction process does not rely on any predefined ontology schema. Hence, it becomes a challenge to derive a unified standard to judge the quality of extractions. Neural-based solutions are evaluated on benchmark datasets [Stanovsky and Dagan, 2016; Bhardwaj et al., 2019].

- **Performance Evaluation:** we first introduce their annotation standards. Common annotation standards include completeness, correctness, and minimality.
 - Similarity rate to the original text
 - Minimality requires the elements of a tuple to be indivisible units.
 - Completeness requires an OpenIE system to extract all information in a sentence.
 - Correctness requires the extracted tuples to be implied from the sentence, and to have meaningful interpretation.
- **Comparison Systems:** We compare DOIEUD against state-of-the-art sequence-labeling systems, COMPACTIE [8], OpenIE6 [5] and M₂OIE [7], and sequence-generation system, ImojIE [6]. We also compare it against traditional non-neural systems designed for extracting compact facts: MinIE [3].
- **Evaluation Datasets and Metrics:** We evaluate the OpenIE systems both **automatically** and **manually** on standardized benchmarks. For automatic evaluation, we first assess all systems with OIE (benchie), Re-OIE2016, carb, Wire57 and CDR datasets. Table 1 Table 2 shows the statistics of the original and processed datasets.
 - We evaluated each system using (AUC) and F1-score(F1).
 - AUC is Compute f1 score, which is defined as the harmonic mean of precision and recall. We convert NaN to zero when f1 score is NaN. This happens when either precision or recall is NaN or when both precision and recall are zero. Its class version is from [torcheval.metrics.functional import multiclass f1 score](#).
 - We report precision (P), recall (R), and F1 computed by scoring functions. However, based on token-level matching of system extractions against ground truth facts, which contains the following matching functions: lexical match, tuple match and similarity match.

Hyperparameters

- We first trained the model for one epoch with an initial learning rate of 0.0001, batch_size=10, epcho=20 and Adam optimization is a stochastic gradient descent method that is based on adaptive estimation of first order and second-order moments.
- We second trained the model for one epoch with an initial learning rate of 5e-5, batch_size=8, epcho=15 and AdamW (Loshchilov and Hutter,2019) was used as an optimizer in combination with training heuristics, such as learning rate warm up (Goyaletal.,2017) and gradient clipping (Pascanuetal.,2013).

Table 2 Number of Sentences extracted relation

Number of sentences extracted relations					
Models\Data sets	OIE (benchie)	Re-OIE2016	carb	Wire57	CDR
DOIEUD	300	591	641	57	500
Graph4nlp	244	480	521	49	499
BERT Relation Extraction master	152	311	334	33	329
benchie (2022)	296	-	-	-	-
benchie_fl (2024)	299	-	-	-	-
CompactIE (2022)	-	-	577	56	-

4.2.2 Experimental Results

4.2.2.1 Performance Evaluation:

The results showed a high similarity between the original text and the extracted relationships compared to the relationships extracted from graph4nlp, as shown in Table 3 and Figure 11 Similarity with sentences.

Table 3 similarity with sentences

	OIE (benchie)	Re-OIE2016	carb	wire57	CDR
DOIEUD	0.89955881	0.901918936	0.90001878	0.91004687	0.833841287
Graph4nlp	0.68857176	0.688189337	0.68356599	0.75218972	0.735735905

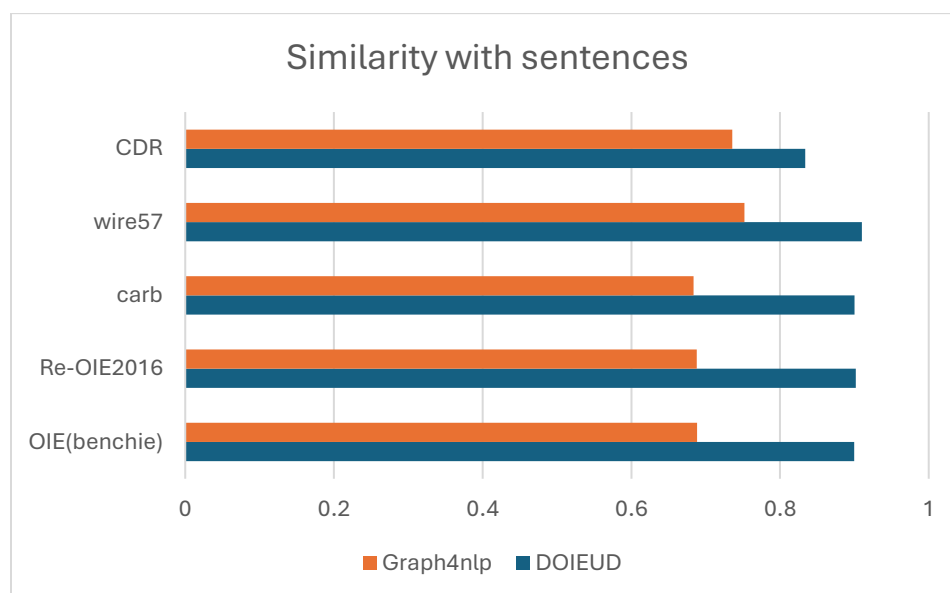


Figure 11 Similarity with sentences

Our full annotation guidelines, along with examples illustrating the principles can be found in Table 4. From the previous table, we noticed that our model largely achieved the basic principles in extracting the three triple relations compared to different models and on different benchmark data.

Table 4 Illustration of annotation principles. The facts in red (preceded by a cross mark) should not.

Principle	Sentence	Model	Annotations
Minimality	Several years later the remaining trackage at Charles City was abandoned.	DOIEUD	the remaining trackage --> Several years later, abandoned, the remaining trackage at --> Charles City
		Clausie	Several years the remaining trackage at Charles City --> was abandoned --> later [×] the remaining trackage at Charles --> Several years was abandoned --> later [×]
		openIe6	the remaining trackage at Charles City --> was abandoned --> Several years later [×]
		reverb	-
		mini	remaining trackage at Charles City --> was abandoned --> Several years later [×]
		m2oie	Several the remaining trackage at Charles City -->was abandoned --> years [×]
		Imojio	the remaining trackage at Charles City --> was abandoned --> Several years later [×]
		compactie	-
		benchie	Cluster 1: [the] remaining trackage [at Charles City] --> was abandoned --> Several years later [the] remaining trackage --> was abandoned --> Several years later [the] remaining trackage [at Charles City] --> was --> abandoned Several years later [the] remaining trackage --> was --> abandoned Several years later [×]
		benchie ^{fl}	Cluster 1: Charles City --> has --> [the] [remaining] trackage Charles City --> had --> [the] [remaining] trackage [the] [remaining] trackage --> is in --> Charles City [the] [remaining] trackage --> was in --> Charles City Cluster 2: [the] [remaining] trackage at Charles City --> was --> abandoned Cluster 3: [the] [remaining] trackage at Charles City --> was abandoned --> [Several] years later [×]

		Graph4nlp	-
Correctness - Completeness	23.8% of all households were made up of individuals and 13.0% had someone living alone who was 65 years of age or older.	DOIEUD	23.8% --> of-->all households 23.8% --> were made up, made of -->individuals 23.8%-->were made --> of all households up of individuals 13.0% -->had-->someone 13.0% -->living alone was-->65 years of age or older 13.0%-->up, made of had-->someone living alone who was 65 years of age or older someone-->living alone was-->65 years of age or older
		Clausie	23.8 % of all households-->were made up-->of individuals 23.8 % of all households-->were made up-->of 13.0 % had someone living alone [×] False Fact and Missing Fact 13.0 %-->be had-->someone living alone 13.0 % had someone living alone-->was-->65 years of age 13.0 % had someone living alone-->was-->65 years of older 13.0 % had someone living alone-->was-->65 years
		openIe6	23.8 % of all households--> were made up-->of individuals 13.0 % -->had-->someone living alone who was 65 years of age someone living alone-->was-->65 years of age 13.0 % -->had -->someone living alone who was older someone living alone-->was-->older
		reverb	23.8 % -->of all households-->were made up of individuals 13.0 % -->had -->someone someone-->was-->65 years of age Missing Fact “Living alone 65 years of age or older”
		mini	23.8 % of households ->were made up of->individuals had someone living alone [×] False Fact 23.8 % of households->were made->13.0 % [×] False Fact someone living alone->was 65 years of->age someone living alone->was 65 years of->older someone living alone->was->65 years
		m2oie	23.8 % of all households->were made up->of individuals and 13.0 % [×] False Fact had someone living alone-> was-> 65 years of age or older
		Imojio	23.8 % of all households-> were made up->of individuals 13.0 %->had->someone living alone who was 65 years of age or older
		compactie	23.8-> up->of individuals Missing Fact “23.8 percentage or number” 23.8 % of all households->were made->of individuals 13.0 %->had->someone living alone Missing Fact “65 years of age or older”
		benchie	Cluster 1: 23.8 % [of all households] --> were made up of --> individuals 23.8 % [of all households] --> were made up --> of individuals 23.8 % [of all households] --> were made --> up of individuals

		23.8 % [of all households] --> were --> made up of individuals 23.8 % [of households] --> were made up of --> individuals 23.8 % [of households] --> were made up --> of individuals 23.8 % [of households] --> were made --> up of individuals 23.8 % [of households] --> were --> made up of individuals Cluster 2: 13.0 % [of all households] --> had --> someone living alone [who was 65 years of age] 13.0 % [of all households] --> had someone living --> alone [who was 65 years of age] 13.0 % [of households] --> had --> someone living alone [who was 65 years of age] 13.0 % [of households] --> had someone living --> alone [who was 65 years of age] Irrelevant fact without the optional [households] We don't know what this percentage applies to Without “was 65 years of age” the fact is not relevant information.
	benchie ^{fl}	Cluster 1: 23.8 % of [all] households --> were made up of --> individuals Cluster 2: 13.0 % of [all] households --> had someone living alone who was --> 65 years of age or older 13.0 % of [all] households --> had someone living alone who was --> 65 years or older 13.0 % of [all] households --> had --> someone living alone who was 65 years or older
	Graph4nlp	%-->had-->someone living alone [×] %-->were made up of-->individuals [×] Missing Fact

4.2.2.2 Automatic Evaluation

Table 5 shows the performance of different OpenIE systems on BenchIE and BenchIE^{FL}. As shown, MinIE outperforms all other systems. We found that MinIE aims to exhaustively produce different representations of the same fact. In contrast, DOIEUD follows the setup of neural OpenIE systems and encourages at most one representation per fact. As a result, MinIE produces extractions than other, achieving much higher recall than its neural counter parts due to Limitations of BenchIE^{FL} and BenchIE for example:

Limited Scope: BenchIE^{FL} and BenchIE may have a limited scope in terms of the linguistic phenomena they can effectively analyze, potentially restricting their applicability to broader or more complex language tasks.

Generalization: These tools may struggle with generalizing to unseen or diverse linguistic patterns, leading to potential inaccuracies or inconsistencies in their analyses.

Complex Sentences: Both BenchIE^{FL} and BenchIE may face challenges in accurately parsing and analyzing complex sentences or structures, potentially leading to errors or incomplete annotations.

Ambiguity Handling: Dealing with ambiguous language constructs or interpretations could be a challenge for BenchIE^{FL} and BenchIE, impacting the accuracy and reliability of their annotations.

Resource Intensive: These tools might be resource-intensive in terms of computation and memory requirements, potentially limiting their usability on devices with constrained resources.

Handling Novel Text: BenchIE^{FL} and BenchIE may struggle with handling novel or out-of-domain text, potentially leading to reduced accuracy or meaningful annotations for such data.

Evaluation Metrics: The evaluation metrics used to assess the performance of BenchIE^{FL} and BenchIE may not fully capture their effectiveness in real-world applications, leading to potential discrepancies between evaluation results and practical utility. Moreover, these benchmarks are incomplete, meaning that the gold extractions do not include all acceptable surface realizations of the same fact. These drawbacks encouraged us to additionally perform fact-centered evaluations.

Table 5 Performances of OpenIE systems on the BenchIE and BenchIE^{FL} dataset

benchie_fl									
System	precision		recall	F1					
reverb	0.440318302		0.091158704	0.151046406					
clausie	0.14713217		0.129599121	0.137810219					
minie	0.286521388		0.196046129	0.232802279					
imojie	0.220238095		0.061504668	0.0961563					
openie6	0.198992443		0.087314662	0.121372873					
m2oie	0.14511041		0.050521691	0.074949084					
compactie	0.172463768		0.06534871	0.094782955					
DOIEUD	0.0493782		0.074135091	0.059275521					
benchie									
	precision			recall			F1		
System	print Slote	print lexical	Similarity (90%)	print Slote	print lexical	Similarity (90%)	print Slote	print lexical	Similarity (90%)
stanford	0.11082	0.11858	0.73529	0.15703	0.16592	0.64102	0.12994	0.13831	0.68493
clausie	0.5029	0.5626	0.9047	0.2555	0.2859	0.487	0.3388	0.3791	0.633
minie	0.429	0.4452	0.9696	0.277	0.274	0.8205	0.3372	0.3392	0.88888
openie6	0.31108	0.41368	0.88571	0.21407	0.28222	0.79487	0.25362	0.33553	0.83783
m2oie	0.39240	0.48188	0.92857	0.16074	0.19703	0.66666	0.22806	0.27970	0.77611
IMoJIE	0.343	-	-	0.128	-	-	0.186	-	-
compactie	0.403	-	-	0.19	-	-	0.262	-	-
graph4nlp	0.1760	0.1879	0.68	0.074	0.078	0.4358	0.104	0.111	0.53125
DOIEUD	0.0261	0.0262	0.5116	0.1585	0.1592	0.5641	0.044	0.045	0.5365

Table 6 summarizes the performance of OpenIE systems across datasets and scoring functions for Wire57, OIE (benchie), CDR, and CaRB. DOIEUD outperforms OpenIE systems by a high percentage and the model performance improves more when using AdamW optimizer instead of Adam except when we use Re-OIE2016 model Multi₂OIE more accuracy than my model due to some types of extractions, such as contextual information cannot probably get it right in our model for example "To allow this massive level of unfettered federal borrowing without prior congressional approval would be irresponsible, " said Rep. Fortney Stark D., Calif., who has introduced a bill to limit the RTC 's authority to issue debt."

False extract Rep. Fortney Stark D. \ would be irresponsible said \ Calif.

True extract Rep. Fortney Stark D.\ Said has introduced \ a bill

Reasons for improvement when using Adamw:

Compatibility with Diverse Architectures:

AdamW is compatible with a wide range of neural network architectures and deep learning tasks, making it a versatile optimization algorithm that can be applied effectively in various scenarios.

State-of-the-Art Performance:

In many deep learning applications, AdamW has shown state-of-the-art performance in terms of convergence speed, generalization, and overall optimization effectiveness, making it a preferred choice for many deep learning tasks.

Utilizing AdamW in deep learning projects can often lead to more stable training, improved generalization, and better control over model complexity, ultimately contributing to enhanced performance and robustness of deep learning models.

Figure 12 wire75 DOIEUD (1) Avg:4.23612557 Min:3.2511247 and Figure 14 CARB DOIEUD (1) Avg:2.366318955 Min:1.701424

CrossEntropyLoss is a commonly used loss function in PyTorch for classification tasks where the model needs to predict one class out of multiple classes. It combines a softmax layer and the negative log-likelihood loss. This loss function is suitable for multi-class classification problems.

The values you provided, Avg: 2.366318955 and Min: 1.701424, seem to be related to the training loss. During the training of a neural network model using CrossEntropyLoss, the loss values represent how well the model is performing at each iteration or epoch.

- **Average Training Loss (Avg: 2.366318955):** This value indicates the average loss over a certain period during the training process. A lower average loss typically indicates that the model is converging well and making accurate predictions on the training data.
- **Minimum Training Loss (Min: 1.701424):** This value represents the lowest loss value achieved during the training process. It signifies the best performance of the model in terms of loss, possibly indicating a point where the model made the most accurate predictions on the training data.

Figure 13 wire75 DOIEUD (2) Avg:3.55247976 Min:1.5615752 Figure 15 CARB DOIEUD (2) Avg:1.684737347 Min:0.7057095 show that DOIEUD (2) better performance of the model DOIEUD (1) by using AdamW optimizer. In general, a lower loss value indicates better performance of the model on the training data.

4.2.2.3 Manual Evaluation

????????????????????

Table 6 Performance of OpenIE systems on Wire57, OIE (benchie), Re-OIE2016, CDR and CaRB datasets.

	Accuracy (F1)				
	wire 75	OIE (benchie)	Re-OIE2016	carb	CDR
DOIEUD (1)	Avg:41 Best:63 (19)	Avg:67 Best:74 (13)	Avg:51 Best:66 (18)	Avg:54 Best :68 (20)	Avg:43 Best:52 (3)
DOIEUD (2)	Avg:71 Best:82(15)		Avg:0.65 Best:0.71(15)	Avg:66 Best:72 (11)	
CompactIE (2022)	31.8	-	-	45	-
imojie (2020)	27	-	-	46.5	-
openie6 (2020)	22.8	-	-	44.4	-
Multi2OIE (2020)	24.1	-	74.6	46.3	-
mini (2017)	31	-	-	41.6	-

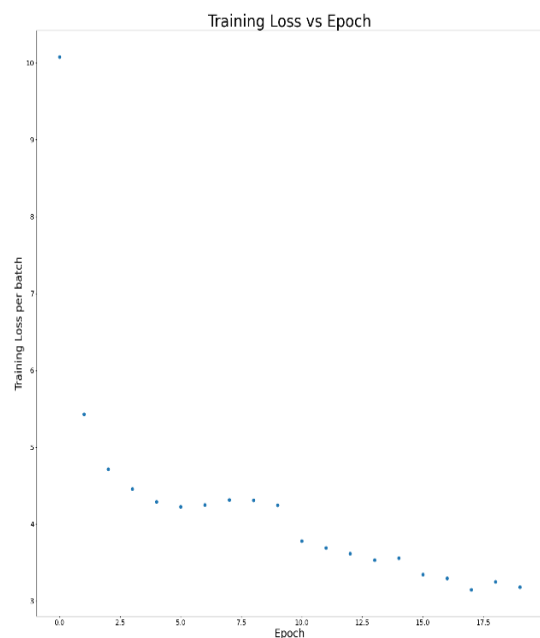


Figure 12 wire75 DOIEUD (1) Avg:4.23612557 Min:3.2511247

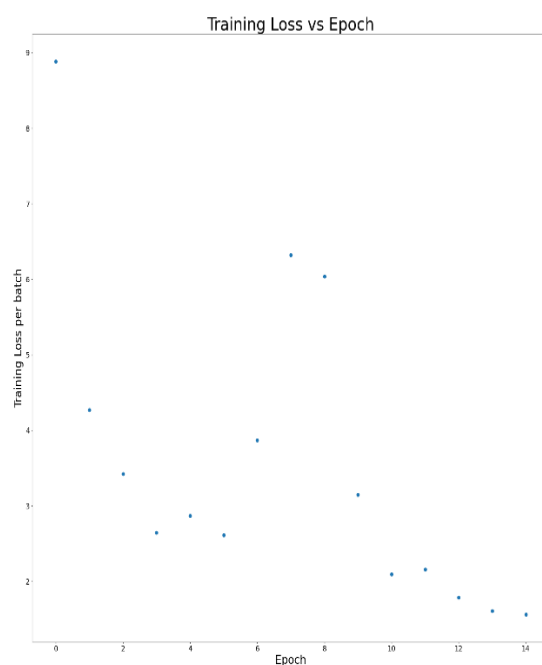


Figure 13 wire75 DOIEUD (2) Avg:3.55247976 Min:1.5615752

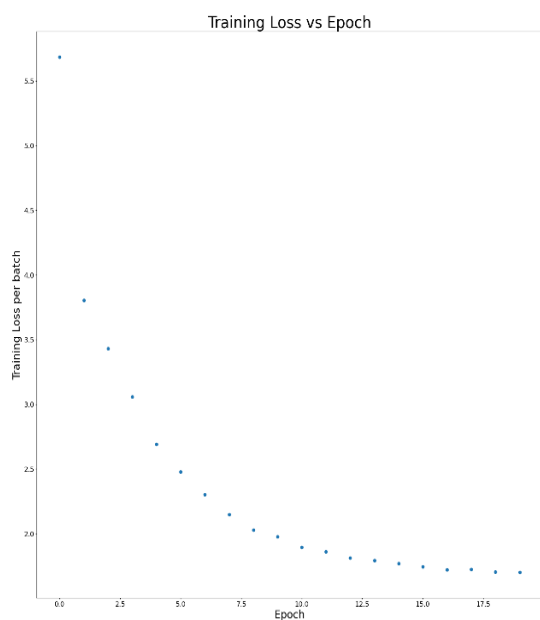


Figure 14 CARB DOIEUD (1) Avg:2.366318955 Min:1.701424

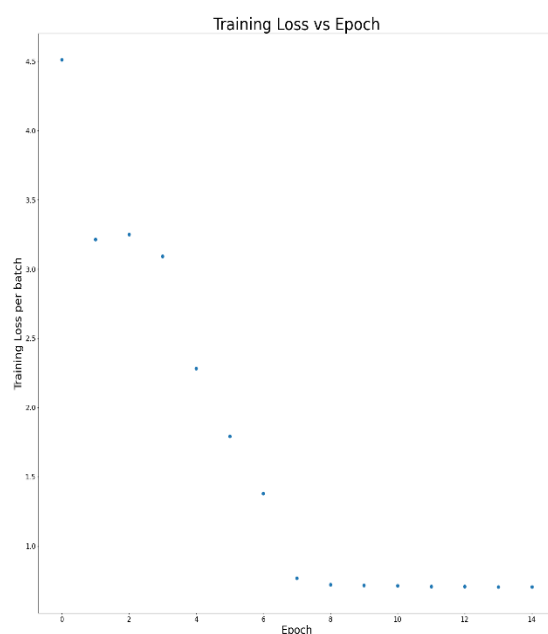


Figure 15 CARB DOIEUD (2) Avg:1.684737347 Min:0.7057095

5 Conclusion

In conclusion, Deep Open Information Extraction from unstructured data (DOIEUD) holds significant promise for automating the extraction of structured information from text at scale. By leveraging deep learning techniques, researchers and practitioners can unlock valuable insights from unstructured data sources, paving the way for advancements in natural language processing, information retrieval, and knowledge discovery.

6 References

1. Anthony Fader, Stephen Soderland, and Oren Etzioni. (2011). Identifying relations for open information extraction. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 1535–1545, Edinburgh, Scotland, UK. Association for Computational Linguistics.
2. Luciano Del Corro and Rainer Gemulla. 2013. Clausie: Clause-based open information extraction. In *Proceedings of the 22nd International Conference on World Wide Web, WWW '13*, page 355–366, New York, NY, USA. Association for Computing Machinery.
3. Kiril Gashteovski, Rainer Gemulla, and Luciano del Corro. 2017. MinIE: Minimizing facts in open information extraction. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2630–2640, Copenhagen, Denmark. Association for Computational Linguistics.
4. Baldini Soares et al., ACL 2019. Matching the Blanks: Distributional Similarity for Relation Learning (<https://aclanthology.org/P19-1279>)
5. Keshav Kolluru, Vaibhav Adlakha, Samarth Aggarwal, Mausam, and Soumen Chakrabarti. 2020a. OpenIE6: Iterative Grid Labeling and Coordination Analysis for Open Information Extraction. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3748–3761, Online. Association for Computational Linguistics.
6. Keshav Kolluru, Samarth Aggarwal, Vipul Rathore, Mausam, and Soumen Chakrabarti. 2020b. IMoJIE: Iterative memory-based joint open information extraction. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5871–5886, Online. Association for Computational Linguistics.
7. Youngbin Ro, Yukyung Lee, and Pilsung Kang. 2020. Multi²OIE: Multilingual open information extraction based on multi-head attention with BERT. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1107–1117, Online. Association for Computational Linguistics.
8. Farima Fatahi Bayat, Nikita Bhutani, and H. Jagadish. 2022. CompactIE: Compact facts in open information extraction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 900–910, Seattle, United States. Association for Computational Linguistics
9. Wu, L., Chen, Y., Shen, K., Guo, X., Gao, H., Li, S., . . . Long, B. (2022). Graph Neural Networks for Natural Language Processing: A Survey. arXiv preprint arXiv:2106.06090.
10. Kiril Gashteovski, Mingying Yu, Bhushan Kotnis, Carolin Lawrence, Mathias Niepert, and Goran Glavaš. 2022. BenchIE: A framework for multi-faceted fact-based open information extraction evaluation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4472–4490, Dublin, Ireland. Association for Computational Linguistics.
11. Fabrice Lamarche and Philippe Langlais. 2024. BenchIE^{FL}: A Manually Re-Annotated Fact-Based Open Information Extraction Benchmark. *Findings of the Association for Computational Linguistics ACL 2024*, pages 8372–8394 August 11-16, 2024, ©2024 Association for Computational Linguistics